Lab Assignment 1: INTRO TO VERILOG & RTL Coding Styles

Part 1: The Philosophy of Hardware Description

1. Why We Use HDLs

• Why did drawing circuits become too difficult as technology advanced?

Because complexity hit aa wall of unimaginable wires which nictitated an upgrade in representing the circuits

• How did using Hardware Description Languages (HDLs) fix this problem?

Hdls fixed this problem by representing the behavior of the circuit using readable code

2. Hardware vs. Software

• What is the difference between an HDL and a software language?

Hdls perform code lines in a parallel manner while software languages mostly perform in a serial manner

• In your answer, explain how each language handles tasks happening at the same time (parallel) versus one after another (serial).

For example if you create some connected gates with inputs and outputs each gate wont wait for the other to output its value rather they all work outputting a value based on their inputs at every instant

If you typed

assign y = x;

assign a = b;

then at every instant in runtime y will take the value of x and a will take the value of b

3. The Evolution of Verilog

• Which two hardware languages were combined to make this standard?

Verilog and system Verilog

• Why was it helpful for engineers to combine these two languages into one?

Verilog was private at first but system Verilog was public so Verilog had to go public to avoid extinction and combining the two in one standard unified the development and focused the effort of engineers on one standard to develop it to the best

Part 2: Navigating the Abstraction Hierarchy

1. The Y-Chart Domains

The Gajski-Kuhn Y-Chart shows three different ways (or domains) to look at a digital system.

• List the names of these three domains.

Behavioural, structural and physical domain

2. Choosing the Right Level

Register-Transfer Level (RTL) is the standard abstraction level used for designing real hardware like FPGAs and chips.

• Why do engineers use RTL for creating hardware instead of the higher Behavioral level? (Hint: Think about what the synthesis tool needs to know).

Because rtl describes how data flows between registers on clock edges and that makes it accurate and most importantly its synthesizable

3. Comparison of Verilog Abstraction Levels.

Verilog supports four primary abstraction levels to describe digital hardware: Behavioral (Algorithmic), Dataflow, Gate, and Structural.

• Compare the Verilog abstraction levels in terms of the primary focus, used Verilog keywords and constructs, typical design suitability, and synthesizability of each level

	Behavioral level	RTL level	Structural level	Gate level
Primary focus	Focuses on how the circuit works, functionalitity not implementation details	Describes data flow between registers on clock edges, cycle accurate	Describes the hierarchy and connection of components, no internal logic mentioned	Describes the physical implementation using basic logic gates, difficult for humans to read
Keywords	Initial, wait, forever, High level c code delays (#5)	Always@(posedge) Non blocking (<=) and blocking(==) Assign	Modules, instantiations, Ports and wires	Primitive gates (AND, NOT, OR) Timing delays and netlists
Design suitability	Test benches and high level simulation	ASICs, FPGAs and IPs	System top level And connecting blocks together	Post synthesis verification and timing analysis
synthesizability	Mostly not synthesizable	Cycle accurate and synthesizable	Synthesizable	Synthesizable